Chapter 12 — Benchmarking and Performance Methodology

> *"The most dangerous number in AI infrastructure is an unverified benchmark. It misleads architecture decisions, justifies wrong hardware purchases, and hides real problems behind confident-sounding percentages. Measure correctly first. Then optimize."* — AI Performance Engineering Principle

---

Chapter Overview

Every optimization in this book produces a number. Whether that number is trustworthy depends entirely on how it was measured. A 40% TFLOPS improvement on a single kernel means nothing if the system is memory-bandwidth bound. A 1.8× throughput gain in a microbenchmark means nothing if it does not survive the production serving stack.

This chapter provides the complete benchmarking and measurement methodology for a Principal Performance Architect. By the end you will be able to apply the five rules of measurement validity, use the Roofline model to classify any operation, read and critique MLPerf results, run the GPU microbenchmark suite, measure training MFU and step-time breakdown, apply the five-step profiling decision tree, and design a performance regression CI pipeline.

Sections in this chapter:

- 12.1 Why Benchmarking Is Hard — The Five Rules of Measurement Validity
- 12.2 The Roofline Model in Practice
- 12.3 GPU Microbenchmarks — Characterizing Raw Hardware
- 12.4 LLM Inference Benchmarks — Prefill, Decode, and Serving
- 12.5 MLPerf — The Industry Standard and Its Limitations
- 12.6 Training Benchmarks — MFU and Step-Time Breakdown
- 12.7 Nsight Systems → Nsight Compute — The Two-Level Profiling Workflow
- 12.8 The Five-Step Profiling Decision Tree
- 12.9 Performance Regression CI Pipeline
- 12.10 The Benchmarking Checklist [r1]
- 12.11 Key Takeaways and Review Questions

12.1 Why Benchmarking Is Hard

Benchmarking a GPU system correctly is significantly harder than it appears. The hardware is asynchronous, thermally dynamic, and shares resources across concurrent processes. Getting a number is easy. Getting a *correct* number requires disciplined methodology.

12.1.1 The Five Rules of Measurement Validity

Every benchmark result that violates one of these five rules should be treated as untrustworthy until the violation is corrected.

```
RULE 1 — WARM UP THE SYSTEM:
  GPU hardware does not operate at peak performance from cold state:
    - GPU clock frequency ramps from low-power P-state to full boost.
    - HBM temperature stabilizes (affects refresh overhead ~3-12%).
    - CUDA JIT compiler finishes first-run kernel compilation.
    - PyTorch allocates and warms up internal memory pools.

  Warm-up protocol:
    Run ≥ 10 iterations of the benchmark before beginning any timing.
    For inference: ≥ 30 seconds of continuous load.
    For training: ≥ 5 steps.
    Confirm warm: GPU temperature stable ± 2°C, power draw stable ± 5W.

  What happens without warmup:
    Cold first-call includes: JIT compilation (50-200ms), memory allocation,
    P-state ramp, and CUDA context initialization.
    A "1-run benchmark" captures all of this as if it were real workload time.
    Numbers will be 20-40% slower than production steady-state.

RULE 2 — MEASURE AT THE SYSTEM BOUNDARY:
  Kernel time ≠ user-perceived latency.
  The gap between GPU kernel execution and wall-clock time includes:
    Python dispatch overhead: 0.1-5ms per call
    CUDA kernel launch overhead: 5-50μs per kernel × hundreds per step
    Memory allocation: variable (avoid by pre-allocating)
    Synchronization barriers: variable

  For throughput: measure wall-clock time over N complete forward passes.
    time_start = time.perf_counter()
    for _ in range(N):
        output = model(inputs)
    torch.cuda.synchronize()  # CRITICAL: ensure GPU work is done
    time_end = time.perf_counter()
    throughput = N / (time_end - time_start)

  For latency: use CUDA events, not Python time:
    start_event = torch.cuda.Event(enable_timing=True)
    end_event   = torch.cuda.Event(enable_timing=True)
    start_event.record()
    output = model(inputs)
    end_event.record()
    torch.cuda.synchronize()
    latency_ms = start_event.elapsed_time(end_event)
```

```
RULE 3 — REPORT PERCENTILES, NOT AVERAGES:
  Average latency hides the tail behavior that destroys user experience.

  Example: P50=42ms, P95=85ms, P99=340ms.
  Average: (42×50 + 85×49 + 340×1) / 100 = 48.8ms

  The average (49ms) looks acceptable.
  The P99 (340ms) violates any realistic SLA.
  Users experiencing P99 will churn.

  For throughput: report mean ± standard deviation over ≥ 30 runs.
  For latency: report P50, P95, P99 over ≥ 100 requests.
  For training: report per-step time P50, P99, and CV (coefficient of variation).

RULE 4 — THERMAL STEADY-STATE:
  GPUs throttle under sustained load when heat exceeds cooling capacity.
  Short-burst benchmarks (< 60 seconds) may reflect pre-throttle performance.
  Production workloads run for hours at sustained load.

  Test protocol: run benchmark for ≥ 10 minutes under sustained load.
  Measure throughput in the LAST 5 minutes (thermal equilibrium).
  Compare to the first 60 seconds: if > 5% degradation, thermal throttling
  is occurring and you must either accept the throttled number as the
  production number, or fix the thermal management (Chapter 5).

RULE 5 — ISOLATE FROM CONCURRENT PROCESSES:
  GPU memory bandwidth is shared across all processes on the GPU.
  A monitoring daemon doing frequent GPU reads will impact benchmarks.
  Docker cgroups can limit CPU bandwidth affecting Python dispatch.

  Before benchmarking:
    Kill all non-essential GPU processes: nvidia-smi | check for unexpected PIDs
    Set GPU to exclusive mode: sudo nvidia-smi -c EXCLUSIVE_PROCESS
    Disable GPU boost variance: sudo nvidia-smi -lgc [base_clock],[base_clock]
    Pin CPU affinity: taskset -c 0,1,2,3 python benchmark.py
```

12.1.2 The Cooldown vs Thermal Steady-State Distinction

```
THERMAL BEHAVIOR PATTERNS:

  PATTERN A — Throttle-then-stabilize (most workloads):
    0-60s:   GPU at boost clock, high performance, building up heat.
```

```
    60-300s: Gradual thermal throttle as heat accumulates.
    300s+:   Steady state at throttled frequency.

    CORRECT BENCHMARK: measure at 300s+ (steady state).
    WRONG: measure at 0-60s. Reports ~10-25% higher than production.

  PATTERN B — No throttle (DLC-cooled systems):
    0-∞:    GPU at full boost, temperature stable at DLC setpoint.
    No throttle observed. Steady state = max performance.

    CORRECT BENCHMARK: measure after 60s warmup.

  CHECKING THERMAL STATE:
    nvidia-smi -q | grep "Thermal Limit"
    nvidia-smi --query-gpu=temperature.gpu,clocks.sm,clocks.mem \
               --format=csv -l 1
    Watch: SM clock should be stable (not decreasing) during benchmark.
    If SM clock decreases after 60s of load: thermal throttle is active.
```

12.2 The Roofline Model in Practice

The Roofline model is the first tool to apply before any optimization. It classifies the bottleneck and tells you which category of optimization matters.

```
ROOFLINE FORMULA:
  Performance (FLOP/s) = min(Peak_FLOPS, AI × HBM_bandwidth)

  AI = Arithmetic Intensity = FLOPs / bytes_of_memory_traffic

  H100 SXM5 parameters:
    Peak BF16 FLOPS:     989 TFLOPS = 989 × 10^12 FLOP/s
    HBM3 bandwidth:      3.35 TB/s  = 3,350 × 10^9 bytes/s
    Ridge point:         989 / 3.35 = 295 FLOP/byte

  DECISION:
    AI < 295: memory-bandwidth-bound → optimize bytes, not FLOPs
    AI > 295: compute-bound → optimize arithmetic efficiency

APPLYING ROOFLINE TO LLM OPERATIONS:

  Operation                      Batch   AI calc           Regime
  ──────────────────────────────────────────────────────────────────
  Decode GEMM (70B)              B=1     ~1 FLOP/byte      Memory-bound
  Decode GEMM (70B)              B=32    ~32 FLOP/byte     Memory-bound
  Decode GEMM (70B)              B=256   ~256 FLOP/byte    Near ridge
  Prefill GEMM (70B, S=2048)     B=1     ~2048 FLOP/byte   Compute-bound
  Prefill GEMM (70B, S=128)      B=1     ~128 FLOP/byte    Memory-bound
  RMSNorm (unfused)              any     ~3-5 FLOP/byte    Memory-bound
  FlashAttention (S=4096)        B=8     ~200-250 FLOP/byte Near ridge
  Large-batch FFN GEMM           B=512   ~512 FLOP/byte    Compute-bound
  ──────────────────────────────────────────────────────────────────

ROOFLINE INTERVIEW RULE:
  For any operation, ask: "How many FLOPs does it do, and how many bytes
  does it load?" If AI < ridge point: buying a faster GPU helps decode
  only if HBM bandwidth is higher, not if TFLOPS are higher.

  Example: H200 vs H100 for decode (70B, batch=1)
    H100: AI ≈ 1 FLOP/byte << 295 ridge. Bottleneck: HBM (3.35 TB/s).
    H200: AI ≈ 1 FLOP/byte << 206 ridge. Same regime. Bottleneck: HBM (4.8 TB/s).

    H200 decode speedup: 4.8 / 3.35 = 1.43× (from BW, not compute).
    FP8 compute on H200: same 989 TFLOPS → zero decode benefit from FP8 TFLOPS.
    Conclusion: always check the regime before claiming a speedup.
```

12.3 GPU Microbenchmarks — Characterizing Raw Hardware

12.3.1 The 30-Minute Hardware Qualification Suite

Run this suite when receiving a new GPU or after major driver/firmware updates. Every number should be within 5% of the reference for a healthy H100.

```python
# ── BENCHMARK 1: GEMM THROUGHPUT ──────────────────────────────────────────
import torch, time

def gemm_benchmark(M=4096, N=4096, K=4096, dtype=torch.bfloat16, n_warmup=10, n_iter
    A = torch.randn(M, K, dtype=dtype, device='cuda')
    B = torch.randn(K, N, dtype=dtype, device='cuda')

    for _ in range(n_warmup):
        C = torch.mm(A, B)
    torch.cuda.synchronize()

    start = torch.cuda.Event(enable_timing=True)
    end   = torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(n_iters):
        C = torch.mm(A, B)
    end.record()
    torch.cuda.synchronize()

    elapsed_ms = start.elapsed_time(end) / n_iters
    flops = 2 * M * N * K  # multiply-add = 2 ops
    tflops = flops / (elapsed_ms * 1e-3) / 1e12

    print(f"GEMM [{M}×{K}]×[{K}×{N}] {dtype}: {tflops:.1f} TFLOPS ({elapsed_ms:.2f}
    return tflops

# Reference: H100 SXM5 BF16, M=N=K=4096 → 850-920 TFLOPS (86-93% of peak)
gemm_tflops = gemm_benchmark()
assert gemm_tflops > 750, f"GEMM underperforming: {gemm_tflops:.0f} < 750 TFLOPS thr
```

```python
```

```python
# ── BENCHMARK 2: HBM BANDWIDTH ──────────────────────────────────────────
def hbm_bandwidth_benchmark(size_gb=1.0, n_warmup=5, n_iters=50):
    size_bytes = int(size_gb * 1024**3)
    src = torch.ones(size_bytes // 4, dtype=torch.float32, device='cuda')
    dst = torch.empty_like(src)

    for _ in range(n_warmup):
        dst.copy_(src)
    torch.cuda.synchronize()

    start = torch.cuda.Event(enable_timing=True)
    end   = torch.cuda.Event(enable_timing=True)
    start.record()
    for _ in range(n_iters):
        dst.copy_(src)  # READ + WRITE = 2× size_bytes BW
    end.record()
    torch.cuda.synchronize()

    elapsed_s = start.elapsed_time(end) / 1000 / n_iters
    bandwidth_tbs = 2 * size_bytes / elapsed_s / 1e12

    print(f"HBM bandwidth ({size_gb:.0f} GB transfer): {bandwidth_tbs:.3f} TB/s")
    return bandwidth_tbs

# Reference: H100 SXM5 → 3.0-3.2 TB/s (90-95% of peak 3.35 TB/s)
hbm_bw = hbm_bandwidth_benchmark()
assert hbm_bw > 2.8, f"HBM underperforming: {hbm_bw:.3f} < 2.8 TB/s threshold"
```

python

```python
# ── BENCHMARK 3: NVLINK P2P BANDWIDTH ──────────────────────────────────
def nvlink_p2p_benchmark(src_gpu=0, dst_gpu=1, size_gb=0.5, n_iters=50):
    with torch.cuda.device(src_gpu):
        src = torch.ones(int(size_gb * 1024**3 // 4), device=f'cuda:{src_gpu}')
    with torch.cuda.device(dst_gpu):
        dst = torch.empty_like(src, device=f'cuda:{dst_gpu}')

    # Warmup
    for _ in range(5):
        dst.copy_(src)
    torch.cuda.synchronize()

    t0 = time.perf_counter()
    for _ in range(n_iters):
        dst.copy_(src)
    torch.cuda.synchronize()
    elapsed_s = (time.perf_counter() - t0) / n_iters

    bw_gbs = size_gb / elapsed_s
    print(f"NVLink GPU{src_gpu}→GPU{dst_gpu}: {bw_gbs:.1f} GB/s")
    return bw_gbs

# Reference: H100 SXM5 NVLink 4.0 → 75-90 GB/s unidirectional (per link pair)
nvlink_bw = nvlink_p2p_benchmark()
assert nvlink_bw > 60, f"NVLink underperforming: {nvlink_bw:.0f} < 60 GB/s"
```

python

```python
# ── BENCHMARK 4: NCCL ALLREDUCE (via nccl-tests, external) ──────────────
# Run from command line (not Python):
# $ mpirun -np 8 ./nccl-tests/build/all_reduce_perf \
#     -b 256M -e 4G -f 2 -g 1

# Expected H100 × 8 (NVLink):
#   256MB: busbw ≥ 160 GB/s
#   1GB:   busbw ≥ 180 GB/s
#   4GB:   busbw ≥ 180 GB/s

# Expected H100 × 8 (InfiniBand NDR 400G):
#   256MB: busbw ≥ 40 GB/s
#   1GB:   busbw ≥ 45 GB/s
#   Alert threshold: busbw < 35 GB/s (degraded IB link)
```

12.3.2 Reference Numbers — H100 SXM5 Healthy System

```
H100 SXM5 HARDWARE REFERENCE CARD (HEALTHY SYSTEM): [SHIPPED]

  COMPUTE:
    GEMM BF16 (4096×4096×4096):    850-920 TFLOPS  (86-93% of peak 989T)
    GEMM FP8 (4096×4096×4096):    1,700-1,850 TFLOPS (86-94% of peak 1979T)
    SFU operations (exp, sin):      ~2.5 TFLOPS (SFUs are 32/SM, not TCs)

  MEMORY:
    HBM3 read bandwidth:            3.0-3.2 TB/s    (90-95% of peak 3.35 TB/s)
    HBM3 write bandwidth:           2.8-3.0 TB/s    (84-90% of peak)
    Measured read+write:            2.8-3.1 TB/s    (copy_perf average)
    L2 bandwidth:                   ~10 TB/s        (when L2 hot)

  INTERCONNECT:
    NVLink unidirectional (p2p):    75-90 GB/s per link pair
    NVLink AllReduce busbw (8GPU):  150-200 GB/s
    InfiniBand NDR 400G busbw:      40-50 GB/s per link (per GPU)

  ALERT THRESHOLDS (trigger investigation):
    GEMM < 750 TFLOPS:  likely alignment issue or wrong precision
    HBM BW < 2.8 TB/s:  likely thermal throttle or memory controller issue
    NVLink p2p < 60 GB/s: NVLink link may be degraded
    IB busbw < 35 GB/s:  InfiniBand link degraded (check ibstatus)
```

12.4 LLM Inference Benchmarks

12.4.1 Prefill and Decode Characterization

```python
```

```python
# LLM INFERENCE BENCHMARK SUITE (using vLLM or direct model)
import time
import numpy as np
import torch
from vllm import LLM, SamplingParams

def benchmark_prefill_latency(model, prompts, num_tokens_to_generate=1):
    """Measure Time-to-First-Token (TTFT) across varying prompt lengths."""
    results = {}

    for prompt_len in [128, 256, 512, 1024, 2048, 4096]:
        # Create synthetic prompt of target length
        prompt = " ".join(["token"] * prompt_len)

        sampling_params = SamplingParams(max_tokens=num_tokens_to_generate)
        latencies = []

        # Warmup
        for _ in range(3):
            model.generate([prompt], sampling_params)

        # Measure
        for _ in range(20):
            t0 = time.perf_counter()
            outputs = model.generate([prompt], sampling_params)
            t1 = time.perf_counter()
            latencies.append((t1 - t0) * 1000)  # ms

        results[prompt_len] = {
            'p50': np.percentile(latencies, 50),
            'p95': np.percentile(latencies, 95),
            'p99': np.percentile(latencies, 99),
            'mean': np.mean(latencies),
            'std':  np.std(latencies),
        }
        print(f"TTFT prompt_len={prompt_len}: P50={results[prompt_len]['p50']:.1f}ms
              f"P99={results[prompt_len]['p99']:.1f}ms")

    return results

def benchmark_decode_tpot(model, prompt, output_lengths):
    """Measure Time-per-Output-Token (TPOT) at different batch sizes."""
    results = {}
```

```
    for batch_size in [1, 4, 16, 32, 64, 128]:
        prompts = [prompt] * batch_size
        sampling_params = SamplingParams(max_tokens=200, min_tokens=200)

        # Warmup
        for _ in range(2):
            model.generate(prompts[:min(4, batch_size)], sampling_params)

        # Measure
        tpots = []
        for _ in range(10):
            t0 = time.perf_counter()
            outputs = model.generate(prompts, sampling_params)
            t1 = time.perf_counter()
            total_output_tokens = sum(len(o.outputs[0].token_ids) for o in outputs)
            tpot_ms = (t1 - t0) * 1000 / total_output_tokens * batch_size
            tpots.append(tpot_ms)

        results[batch_size] = {
            'p50_ms': np.percentile(tpots, 50),
            'p99_ms': np.percentile(tpots, 99),
        }
        print(f"TPOT batch={batch_size}: P50={results[batch_size]['p50_ms']:.1f}ms/t

    return results
```

12.4.2 The Throughput-Latency Curve Sweep

```
THROUGHPUT-LATENCY CURVE METHODOLOGY:

  The single most important characterization of a serving system.
  Reveals: maximum sustainable throughput, latency cliff location,
           optimal operating point, and SLA boundary.

  SWEEP PROCEDURE:
    1. Set target request rate (RPS): [1, 2, 5, 10, 20, 50, 80, 100, 120, ...]
    2. Sustain each rate for ≥ 60 seconds (steady state)
    3. Record: actual throughput (tok/s), P50/P95/P99 TTFT, P50/P99 TPOT
    4. Identify: the "knee" where P99 TTFT starts diverging from P50

  EXPECTED CURVE SHAPE:
    Low RPS:   P99 ≈ P50 (no queuing), GPU underutilized
```

```
    Sweet spot: P99 ≈ 2-3× P50 (light queuing), ~70-80% GPU utilization
    Knee:      P99 starts growing non-linearly (M/M/1 queue entering unstable
zone)
    Cliff:     P99 diverges → queue growing → system overloaded

  CAPACITY METRIC: Maximum server-scenario QPS where P99 TTFT ≤ SLA target
    This is the number that maps directly to: "How many of these servers do I
need?"

  REFERENCE (LLaMA-3 70B, H100×8, vLLM FP8):
    Sustainable throughput: ~6,500-7,000 tok/s (offline, max batch)
    Max server QPS (P99 TTFT ≤ 500ms): ~80-100 req/s at 200-output avg
    Optimal operating point: ~65-75% GPU KV utilization
```

12.4.3 KV Cache Pressure and Admission Control

```
KV PRESSURE CHARACTERIZATION:

  As KV utilization increases, three metrics degrade:
    1. TTFT P99:  increases as scheduler admits requests more slowly
    2. TPOT:      increases as PagedAttention contends for physical blocks
    3. Preemption rate: sequences evicted-and-requeued → latency spikes

  KV pressure sweep:
    Gradually increase concurrent sessions from 20% to 95% KV utilization.
    Record P99 TTFT at each utilization level.
    Identify: "cliff" — utilization level where P99 TTFT jumps non-linearly.

  Expected cliff for LLaMA-3 70B on H100 with PagedAttention:
    At 85% KV utilization: P99 TTFT begins to spike
    At 90% KV utilization: P99 TTFT regularly 3-5× P50 (SLA breach)
    At 92%+ KV utilization: cascading evictions, system degradation

  ADMISSION CONTROL RULE:
    Set admission threshold at 82% KV utilization.
    Return HTTP 429 when threshold exceeded.
    Never operate at > 85% sustained KV utilization.

  vLLM monitoring:
    vllm:gpu_cache_usage_perc → alert at > 0.82
    vllm:num_preemptions_total → alert on any non-zero rate
```

12.5 MLPerf — The Industry Standard and Its Limitations

12.5.1 MLPerf Structure

```
MLPERF INFERENCE BENCHMARK STRUCTURE:

  MODELS (v4.0 / v5.0 suite):
    Llama-2 70B:   BFloat16, server + offline scenario
    Llama-3.1 405B: Server + offline [v5.0+]
    GPT-J 6B:      Summarization (CNN/DailyMail)
    BERT-large:    Question answering (SQuAD)
    ResNet-50:     Image classification (ImageNet)
    DLRM v2:       Recommendation (Criteo)
    Stable Diffusion: Image generation

  SCENARIOS:
    Server:   Arrive as Poisson process. Target: max QPS at P99 latency ≤ SLA.
              70B Llama-2: P99 TTFT ≤ 2s, TPOT ≤ 200ms.
    Offline:  All queries arrive at once. Maximize throughput (tok/s).

  DIVISIONS:
    Open:   Any hardware, any software, any model changes (pruning, quantization,
            distillation allowed as long as accuracy targets met).
    Closed: Restricted optimizations (primarily precision — FP8, INT4 allowed;
            model weights must be functionally equivalent; same architecture).

  READING A MLPERF RESULT:
    Submitter:    Hardware vendor or cloud provider
    System:       GPU model, count, software stack version
    Model:        Llama-2 70B
    Scenario:     Server
    Metric:       X.XX queries/second at P99 latency constraint
    Division:     Closed (more comparable across submitters)

  EXAMPLE (H100 × 8, Llama-2 70B, Server, Closed, v4.0):
    Result: ~3,200 tokens/sec sustained at P99 TPOT ≤ 200ms
    Comparison baseline: ~11,000 tok/s in Offline (no latency constraint)
    Ratio: Server/Offline ≈ 29% — latency constraint cuts throughput 71%.
```

12.5.2 MLPerf Limitations for Production Planning

```
WHAT MLPERF DOES NOT MEASURE (production blind spots):
```

```
1. COLD START AND WARMUP LATENCY:
   MLPerf runs hot with pre-warmed serving stack.
   Production: cold-start latency (GPU activation, model loading) may be
   30-90 seconds. Critical for auto-scaling or spot instance workloads.

2. MULTI-TENANCY AND ISOLATION:
   MLPerf: single model, dedicated hardware.
   Production: multiple models sharing GPU (if using multi-model serving),
   interference from monitoring daemons, background processes.

3. LONG-SESSION DEGRADATION:
   MLPerf: typically short benchmark window (15-30 minutes).
   Production: sessions run for hours. KV cache fills over time.
   After 8+ hours: GC pauses, memory fragmentation, OOM risk on stale seqs.

4. REAL TRAFFIC DISTRIBUTIONS:
   MLPerf: synthetic Poisson arrivals, fixed prompt/output length.
   Production: bursty traffic (business hours vs off-hours), power-law length
   distributions, mix of short/long prompts from real users.

5. TOTAL COST OF OWNERSHIP:
   MLPerf reports throughput, not $/1M tokens.
   A 40% faster GPU may cost 60% more → worse TCO.
   Always convert to cost-per-token for purchasing decisions.

THE CORRECT USE OF MLPERF:
  ✓ Comparing equivalent hardware at equivalent precision (closed division)
  ✓ Tracking improvements across software stack versions on same hardware
  ✓ Initial shortlisting of hardware candidates for further evaluation
  × Direct mapping to production capacity planning
  × Cost-per-token comparison (power not included)
  × Predicting P99 latency under real workload distribution
```

12.6 Training Benchmarks — MFU and Step-Time Breakdown

12.6.1 MFU Measurement

```python
```

```python
def compute_mfu(
    tokens_per_second: float,   # global batch, observed throughput
    model_params: int,          # total parameters
    num_gpus: int,              # GPU count in job
    gpu_peak_flops: float,      # GPU peak BF16 FLOPS (989e12 for H100)
    include_backward: bool = True,
) -> float:
    """
    FLOPs per token (dominant GEMM term):
      Forward:  6 × N_params  (each param used in one MMA = ~2 FLOPs per token)
      Backward: 4 × N_params  (gradient computation is ~2× forward dominant terms)
      Total:    6 × N_params  (forward=2, backward=4 — standard approximation)

    The 6× factor: forward GEMM = 2 FLOPs/param, backward = 4 FLOPs/param.
    This is the Chinchilla / PaLM paper formula; widely adopted.
    """
    multiplier = 6 if include_backward else 2
    flops_per_token = multiplier * model_params

    achieved_flops = tokens_per_second * flops_per_token
    peak_flops_cluster = num_gpus * gpu_peak_flops

    return achieved_flops / peak_flops_cluster

# Example: LLaMA-3 70B, H100×512, 7,000 tokens/sec global
mfu = compute_mfu(7000, 70e9, 512, 989e12)
print(f"MFU: {mfu:.1%}")  # Expected: ~58.8%

MFU INTERPRETATION GUIDE:
  > 65%:    Exceptional — publication-quality efficiency
  55-65%:   Good — well-tuned distributed training (target for H100 BF16)
  40-55%:   Acceptable — room for improvement in overlap/scheduling
  25-40%:   Investigate — likely AllReduce exposure, pipeline bubbles, or data loadi
  < 25%:    Critical problem — significant undiagnosed bottleneck
```

12.6.2 Step-Time Breakdown Instrumentation

python

```python
import torch, time
from contextlib import contextmanager

@contextmanager
def cuda_timer(name: str, timings: dict):
    """Context manager for CUDA-accurate timing."""
    start = torch.cuda.Event(enable_timing=True)
    end   = torch.cuda.Event(enable_timing=True)
    start.record()
    yield
    end.record()
    torch.cuda.synchronize()
    timings[name] = start.elapsed_time(end)  # milliseconds

def instrument_training_step(model, batch, optimizer):
    """
    Detailed step breakdown for bottleneck identification.
    Run 10+ times and take the steady-state median.
    """
    timings = {}

    with cuda_timer("data_to_gpu", timings):
        batch = {k: v.cuda(non_blocking=True) for k, v in batch.items()}

    with cuda_timer("forward", timings):
        loss = model(**batch).loss

    with cuda_timer("backward", timings):
        loss.backward()

    # For DDP/FSDP: AllReduce happens during/after backward.
    # Add explicit barrier to separate communication from compute:
    with cuda_timer("allreduce_wait", timings):
        # In DDP: gradient sync happens here
        torch.distributed.barrier() if torch.distributed.is_initialized() else None

    with cuda_timer("optimizer_step", timings):
        optimizer.step()
        optimizer.zero_grad()

    # Report as percentages
    total = sum(timings.values())
    print(f"\nStep breakdown (total={total:.1f}ms):")
```

```
    for name, ms in sorted(timings.items(), key=lambda x: -x[1]):
        print(f"  {name:20s}: {ms:6.1f}ms  ({100*ms/total:4.1f}%)")

    return timings

# EXPECTED BREAKDOWN (LLaMA-3 70B, H100×8, TP=8, DDP, BF16, batch=4, seq=4096):
# forward:          450ms  (23%)
# backward:         900ms  (46%)
# allreduce_wait:   450ms  (23%)   ← target: < 15%, overlap this!
# optimizer_step:    80ms   (4%)
# data_to_gpu:       70ms   (4%)
# TOTAL:          ~1,950ms
# Note: if AllReduce appears AFTER backward in nsys trace → overlap not working!
```

12.6.3 MFU Tracking in Production

```
MFU LOGGING AND ALERTING SETUP:

  Log per-step MFU to W&B / MLflow:
    wandb.log({"mfu": mfu, "step_time_ms": step_time, "tokens_per_sec": tps})

  ALERT RULES:
    Rolling 10-step MFU drop > 5% from baseline → page on-call
    Single-step MFU drop > 15% → log warning (one-time events acceptable)
    MFU sustained below 35% for > 10 steps → trigger diagnostic script

  DIAGNOSTIC RESPONSE:
    1. Check DCGM for XID errors, thermal throttle, ECC events.
    2. Run AllReduce timing with NCCL_DEBUG=INFO.
    3. Check per-node step time variance (one slow node = entire cluster slows).
    4. Check IB link speed (ibstatus on all nodes).
    5. Inspect nsys timeline for communication overlap.

  Expected MFU benchmarks (LLaMA-3 70B, BF16):
    Single GPU:    65-78% (no communication overhead)
    8-GPU DDP:     60-72% (NVLink, well-tuned overlap)
    64-GPU:        52-65% (IB TP=8 PP=4 DP=2)
    512-GPU:       45-60% (production, full 3D parallel)
    Alert threshold: < 40% at any scale (investigate immediately)
```

12.7 Nsight Systems → Nsight Compute: The Two-Level Workflow

Chapter 7 covered the kernel-level workflow in detail. This section provides the principal architect's view: how to scope the problem before going deep.

12.7.1 Nsight Systems — The System-Level View (First)

```
NSIGHT SYSTEMS — WHAT TO LOOK FOR:

  Command:
    nsys profile \
      --trace=cuda,nvtx,nccl,osrt \
      --sample=cpu \
      --output=profile_%p \
      python training_script.py

  READING THE TIMELINE (in order of importance):

  1. LOOK FOR GAPS between GPU CUDA kernels:
     Gaps > 1ms: overhead-bound (kernel launch, Python dispatch, scheduling)
     Uniform dense blocks: good — GPU computing continuously
     Periodic gaps: often NCCL AllReduce on separate stream (check overlap)

  2. LOOK FOR NCCL STREAM OVERLAP:
     Correct: NCCL kernels appear BELOW backward kernels (same time, different
stream)
     Wrong:   NCCL kernels appear AFTER backward kernels (sequential — overlap
failed)
     Impact: sequential AllReduce adds 25-50% to step time

  3. LOOK FOR STEP-TIME VARIANCE:
     Healthy: all steps same duration ± 3%
     Problem: occasional 2-5× longer steps (stragglers, GC pauses, data loading)
     Diagnosis: zoom into the long step; identify which phase expanded

  4. LOOK FOR CPU OVERHEAD:
     "CPU gap" between GPU kernel completions: Python execution, dispatch
     Consistent > 1ms CPU gaps: torch.compile or CUDA Graphs opportunity
     nsys shows Python call stacks in CPU thread lane

  5. LOOK FOR DATA LOADING GAPS:
     Gaps before each forward pass where GPU is idle: data loading stall
     Fix: increase DataLoader num_workers, enable pin_memory
```

12.7.2 Moving from nsys to ncu

```
WHEN TO USE NSIGHT COMPUTE:

  nsys identifies WHICH kernel is slow and HOW MUCH of total time it consumes.
  ncu explains WHY that specific kernel is slow (hardware counters).

  Only use ncu when:
    1. nsys confirms a specific kernel is > 10% of total time.
    2. The kernel's behavior seems wrong (low TC utilization, high HBM traffic).
    3. You have a specific optimization hypothesis to test.

  NEVER use ncu without nsys first:
    ncu slows execution by 100-1000× (full hardware counter collection).
    Profiling the wrong kernel for hours is a common mistake.

  KEY ncu METRICS REFERENCE (Chapter 7 detail, summary here):
    sm__pipe_tensor_cycles_active:  TC utilization % → < 50% = alignment or fusion
issue
    dram__bytes_read:               HBM reads → vs theoretical minimum
    smsp__warp_issue_stall_*:       Dominant stall type → points to specific fix
    launch__occupancy:              Register / SMEM limiter analysis
```

12.8 The Five-Step Profiling Decision Tree

This framework converts an ambiguous performance symptom into a targeted action in under 30 minutes. Use it before opening ncu.

```
5-STEP PROFILING DECISION TREE:

  SYMPTOM: System is slower than expected.

  STEP 1: MFU / System-Level Metric Check
    Training: compute current MFU (§12.6.1)
    Inference: compute TPOT vs theoretical minimum (model_bytes / HBM_BW)

    MFU > 55% (training):   HEALTHY — look elsewhere if users complain
    MFU 40-55%:             ACCEPTABLE — some room for improvement
    MFU < 40%:              INVESTIGATE → continue to step 2

  STEP 2: SM Utilization Check
    dcgmi dmon -e 1003 -d 1  (SM utilization %)
    nsys timeline: are GPU kernels dense or gapped?

    SM util < 60% during active step:  OVERHEAD/COMMUNICATION BOUND
      → Check AllReduce timing (NCCL_DEBUG=INFO)
      → Check CPU dispatch gaps in nsys (Python overhead)
      → Apply: CUDA Graphs, torch.compile, gradient overlap

    SM util > 60%: GPU is busy → continue to step 3

  STEP 3: HBM Bandwidth Utilization
    dcgmi dmon -e 1005 -d 1  (memory bandwidth utilization %)

    HBM util < 40%:  COMPUTE-BOUND
      → GPU is computing, not waiting on memory
      → Check: are Tensor Cores active? (ncu sm__pipe_tensor_cycles)
      → Check: is precision correct? (BF16/FP8 vs FP32)
      → Apply: FP8 quantization, tile size tuning

  STEP 4: HBM Bandwidth Over 80%
    HBM util > 80%:  MEMORY-BANDWIDTH-BOUND (expected for decode)
      → This is correct for inference decode at small batch
      → Check: is there unnecessary HBM traffic? (un-fused ops)
      → Apply: operator fusion, quantization, increase batch size

  STEP 5: AllReduce Timing
    NCCL_DEBUG=INFO → check AllReduce durations
    nsys → NCCL stream position (overlapped or sequential?)

    AllReduce > 15% of step time:  COMMUNICATION BOTTLENECK
```

```
      → Verify FSDP/DDP overlap enabled (backward_prefetch)
      → Verify bucket sizes (bucket_cap_mb = 25-50)
      → Verify TP within NVLink domain (never across IB)
      → Run nccl-tests to verify IB bandwidth
      → Consider gradient compression for slow networks

STEP SUMMARY TABLE:
  Step  Question                    Low → Problem                High → State
  ──────────────────────────────────────────────────────────────────────────
  1     MFU / Throughput vs target? < 40% MFU → investigate      ≥ 55% → healthy
  2     SM Utilization?             < 60% → overhead/comm bound
  3     HBM bandwidth?              < 40% → compute-bound (check TCs)
  4     HBM bandwidth?              > 80% → memory-bound (normal decode)
  5     AllReduce % of step time?   > 15% → communication bottleneck
```

12.9 Performance Regression CI Pipeline

A performance regression CI pipeline is not a luxury. It is the infrastructure that protects months of optimization work from being silently erased by a driver update.

12.9.1 Three-Tier Benchmark Suite

```
CI BENCHMARK TIERS — BY COMMIT TYPE:

TIER 1 — Every commit (runs in < 5 minutes):
  ├── GEMM microbenchmark (3 representative shapes)
  |     Accept: < 3% deviation from baseline TFLOPS
  ├── HBM bandwidth test (1 GB transfer)
  |     Accept: > 85% of peak baseline
  ├── Kernel launch overhead
  |     Accept: < 5 μs per launch, < 3% deviation
  └── FlashAttention forward (batch=8, seq=2048)
        Accept: < 3% deviation in latency

TIER 2 — Commits affecting kernels, drivers, or framework (< 20 min):
  ├── All Tier 1 tests
  ├── LLM prefill latency sweep (seq=128, 512, 2048, 4096)
  |     Accept: < 3% TTFT regression at P95
  ├── LLM decode throughput (batch=1, 16, 64)
  |     Accept: < 3% TPOT regression at P95
  ├── Single-GPU training step (100 steps)
  |     Accept: < 3% step time regression
```

```
  └── MFU single-GPU sanity check
        Accept: > 55% MFU for standard training shape

TIER 3 — Nightly / Release gate (1-2 hours):
  ├── All Tier 1 + 2 tests
  ├── Multi-GPU AllReduce (8-GPU, 256MB-4GB sweep)
  |     Accept: < 5% bus BW regression
  ├── End-to-end training MFU (8-GPU, 100 steps)
  |     Accept: > 55% MFU, < 3% step time regression
  ├── LLM serving throughput-latency curve (10 request rates)
  |     Accept: < 5% throughput regression at server QPS
  └── 8-GPU scaling efficiency
        Accept: > 88% scaling efficiency
```

12.9.2 Statistical Baseline and Regression Detection

```python
```

```python
import numpy as np
from scipy import stats
from typing import List, Optional

class PerformanceBaseline:
    def __init__(self, metric_name: str, window: int = 20):
        self.metric_name = metric_name
        self.history: List[float] = []
        self.window = window

    def update(self, value: float):
        self.history.append(value)
        if len(self.history) > self.window:
            self.history.pop(0)

    def check_regression(
        self,
        new_value: float,
        sigma_threshold: float = 2.0,
        absolute_threshold: float = 0.03
    ) -> dict:
        """Check for statistically significant regression."""
        if len(self.history) < 5:
            return {"status": "insufficient_baseline"}

        baseline_mean = np.mean(self.history)
        baseline_std  = np.std(self.history)
        z_score = (new_value - baseline_mean) / (baseline_std + 1e-9)
        pct_change = (new_value - baseline_mean) / baseline_mean

        # For throughput metrics: negative z-score = regression
        # For latency metrics: positive z-score = regression
        is_lower_better = "latency" in self.metric_name.lower()
        regression_z = z_score if is_lower_better else -z_score

        result = {
            "metric": self.metric_name,
            "baseline_mean": baseline_mean,
            "baseline_std":  baseline_std,
            "new_value":     new_value,
            "z_score":       z_score,
            "pct_change":    pct_change,
            "is_regression": False,
```

```
            "severity":      None,
        }

        if regression_z < -sigma_threshold:
            result["is_regression"] = True
            if abs(pct_change) > 0.05:
                result["severity"] = "P0"   # Block merge, page oncall
            elif abs(pct_change) > absolute_threshold:
                result["severity"] = "P1"   # Block merge, notify author
            else:
                result["severity"] = "P2"   # Warning, don't block

        return result

# USAGE IN CI PIPELINE:
baseline = PerformanceBaseline("decode_throughput_toks", window=20)
# ... load historical values from artifact storage ...

tonight_result = 10700  # tok/s
report = baseline.check_regression(tonight_result)

if report["is_regression"]:
    z = report["z_score"]
    pct = report["pct_change"] * 100
    print(f"REGRESSION DETECTED: {report['metric']}")
    print(f"  Baseline mean: {report['baseline_mean']:.0f}")
    print(f"  Tonight:       {tonight_result:.0f} ({pct:+.1f}%,  z={z:.2f})")
    print(f"  Severity:      {report['severity']}")
    # Trigger: auto-profile + block PR (P0/P1) or warn (P2)
```

12.9.3 CI Alert Matrix

```
REGRESSION ALERT MATRIX:

  Severity  Metric                      Threshold    Action
  ──────────────────────────────────────────────────────────────────────
  P0        Training throughput         > 5% drop    Block merge, page oncall
  P0        GEMM TFLOPS                 > 5% drop    Block merge, page oncall
  P0        HBM bandwidth               > 5% drop    Block merge, hardware check
  P1        Training throughput         3-5% drop    Block merge, notify author
  P1        P99 TTFT (inference)        > 5% increase Block merge, notify author
  P1        P99 TPOT (inference)        > 5% increase Block merge, notify author
  P1        MFU (training)              > 3% drop    Block merge, notify author
```

```
  P2        AllReduce bus bandwidth    > 5% drop    Warning in PR, no block
  P2        Scaling efficiency (8 GPU) > 3% drop    Warning in PR, no block
  Info      Kernel launch overhead     > 3% increase Log only, no block
  ──────────────────────────────────────────────────────────────────────────


  Auto-generate Nsight Compute trace:  Any P0 or P1 regression
  Retain ncu artifact:                 30 days (linked in PR comment)
  Notify oncall:                        P0 only (PagerDuty integration)
  Post PR comment:                      All P0, P1, P2 regressions with artifact
links
```

12.10 The Benchmarking Checklist [r1]

Before publishing or acting on any benchmark result, verify all 10 items.
If any item is unchecked: the number is not publishable.

```
BENCHMARKING VALIDATION CHECKLIST (10 Items):

BEFORE COLLECTING DATA:
  □ 1. Warmup: ≥ 10 iterations (or ≥ 30s) before timing begins.
         Verify: GPU temperature stable ± 2°C, power draw stable ± 5W.

  □ 2. Thermal steady-state: run for ≥ 5 minutes of sustained load.
         Verify: SM clock not decreasing over time (check with nvidia-smi -l 1).
         If thermal throttle detected: either fix cooling or report throttled
number.

  □ 3. CPU governor: set to "performance" mode.
         Command: sudo cpupower frequency-set -g performance
         Verify: cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
         C-states that introduce latency variance can skew P99 by 5-20ms.

  □ 4. NUMA pinning: process on same NUMA node as target GPU.
         Command: numactl --cpunodebind=0 --membind=0 python benchmark.py
         (GPU 0 typically on NUMA node 0; check nvidia-smi topo -m)

  □ 5. Synchronization: using CUDA events (NOT Python time.perf_counter).
         torch.cuda.synchronize() before/after timed region.
         CUDA events capture GPU execution time, not Python wall-clock.

DURING DATA COLLECTION:
  □ 6. Sample count: ≥ 100 measurements for latency; ≥ 30 for throughput.
         Statistical stability: std/mean < 5% for throughput, < 10% for P99
latency.

  □ 7. Precision is production-correct: using BF16 if production is BF16.
         Never benchmark FP8 and report as BF16. Never use FP32 "for safety."

  □ 8. Batch size matches production: if production serves batch=32,
         benchmark at batch=32, not batch=1 (regime difference!).

REPORTING:
  □ 9. Statistics: P50, P95, P99 for latency; mean ± std for throughput.
         NEVER report only mean latency for a user-facing metric.

  □ 10. Complete context:
          GPU model and count, base/boost clock, peak spec, achieved %
          Software: framework version, driver version, CUDA version
          Precision: BF16/FP8/INT4
```

```
          Batch size, sequence length, model configuration
          Date (hardware/software performance changes over time)
```

H100 SXM5 Principal Architect Quick Reference Card

```
H100 SXM5 PERFORMANCE REFERENCE — PRINCIPAL ARCHITECT CARD: [SHIPPED]

PEAK HARDWARE:
  BF16 GEMM (Tensor Core):     989 TFLOPS (dense)
  FP8 GEMM (Transformer Eng):  1,979 TFLOPS
  HBM3 bandwidth:               3.35 TB/s
  NVLink 4.0:                   900 GB/s/GPU
  Ridge Point (BF16 dense):     ~295 FLOP/byte

EXPECTED ACHIEVED (PRODUCTION):
  GEMM (large, BF16):            850-920 TFLOPS (86-93% of peak)
  HBM bandwidth:                 2.85-3.1 TB/s  (85-93%)
  NVLink AllReduce (8GPU):       150-200 GB/s bus BW

TRAINING MFU RANGES:
  Single GPU (no comm):         65-78%    ← achievable with good kernel stack
  8-GPU NVLink DDP:             60-72%    ← well-tuned with overlap
  64-GPU 3D parallel (IB):      52-65%    ← typical production
  512-GPU (full 3D parallel):   45-60%    ← good cluster config
  Alert threshold:               < 40% MFU at any scale

INFERENCE TYPICAL (LLaMA-3 70B BF16, H100×8):
  Prefill TTFT (S=2048, batch=1):   280-350 ms
  Decode TPOT (batch=1):            40-55 ms  (BW-bound, ~140 GB/step)
  Decode TPOT (batch=32):           45-65 ms  (still BW-bound)
  Max server QPS (P99 TTFT ≤ 500ms): ~80-100 req/s
  Peak offline throughput:           ~11,000 tok/s

ALERT THRESHOLDS (investigate immediately):
  GEMM < 750 TFLOPS:    alignment or wrong precision
  HBM BW < 2.8 TB/s:   thermal throttle or memory controller
  NVLink p2p < 60 GB/s: degraded NVLink link
  IB busbw < 35 GB/s:   degraded InfiniBand link
  MFU < 40%:            significant bottleneck (communication/pipeline)
```

12.11 Key Takeaways and Review Questions

```
CHAPTER 12 CORE PRINCIPLES:

  1. WARMUP + PERCENTILES + STEADY-STATE = THE MEASUREMENT PROTOCOL.
     These three rules are non-negotiable. Every published benchmark
     that skips any of them is suspect. Always measure at production conditions.

  2. ROOFLINE BEFORE OPTIMIZATION — ALWAYS.
     AI < ridge point → memory-bound; AI > ridge → compute-bound.
     The correct optimization depends entirely on which regime you're in.
     Never propose an optimization before applying Roofline.

  3. MFU IS THE NORTH STAR METRIC FOR TRAINING.
     It captures all inefficiencies — communication, pipeline bubbles,
     data loading, thermal throttle — in one number.
     If MFU < 40%: there is a significant undisclosed bottleneck.
     If MFU > 60%: the cluster is well-configured.

  4. NSYS BEFORE NCU — ALWAYS.
     nsys (minutes) identifies which kernel is the bottleneck.
     ncu (hours) explains why that kernel is slow.
     Using ncu without nsys first wastes hours on the wrong kernel.

  5. MLPERF IS FOR COMPARISON, NOT PRODUCTION PLANNING.
     It doesn't measure cold-start, multi-tenant interference,
     real traffic distributions, or cost-per-token.
     Use it for shortlisting. Run your own stack on your workload.

  6. A REGRESSION CI PIPELINE IS NOT A LUXURY.
     A silent 5% MFU regression from a driver update = $1M wasted
     per year on a 10K-GPU cluster. Detect it in the PR, not 6 months later.
     Three tiers: < 5min per commit, < 20min for kernel changes, nightly full.

  7. THE FIVE-STEP DECISION TREE CONVERTS SYMPTOM TO ACTION.
     MFU → SM util → HBM BW (low) → HBM BW (high) → AllReduce %.
     Work through in order. Each step narrows the diagnosis.
```

Review Questions

Difficulty: Foundational

Question 1 (Roofline Application): A new team member claims their PyTorch optimization improved throughput by 32% on an A100 GEMM benchmark (large batch GEMM, batch=256). They want to apply this to improve LLaMA-3 70B decode speed (batch=1) on A100. (a) Classify

large-batch GEMM and decode-batch=1 on the A100 roofline (BF16: 312 TFLOPS, HBM: 2.0 TB/s). Show arithmetic intensity calculations. (b) Explain why a GEMM compute optimization will or will not help decode. (c) What is the correct optimization target for decode throughput?

Expected answer: (a) A100 ridge = 312T/2.0T = 156 FLOP/byte. Large-batch GEMM (batch=256, 70B): AI ≈ 256 FLOP/byte > 156 → compute-bound ✓. Decode (batch=1, 70B): AI ≈ 1 FLOP/byte << 156 → deeply memory-bound ✓. (b) The 32% GEMM improvement is on a compute-bound workload. Decode at batch=1 is memory-bandwidth-bound — its throughput is limited by HBM bandwidth, not by how fast GEMM executes. Even if GEMM were 100× faster, decode throughput would improve < 1% (the compute time is already tiny relative to HBM load time). Applying the optimization to decode = zero benefit. (c) For decode: (1) Quantization — INT4 reduces model bytes from 140 GB to 35 GB → 4× decode throughput. (2) Larger batch size — at batch=156, AI crosses ridge point and compute becomes the bottleneck. (3) Better HBM bandwidth hardware (H100, H200). These are the correct levers for a memory-bound workload.

Difficulty: Intermediate

Question 2 (MFU Diagnosis): A 512-GPU training job reports 38% MFU. Profiling shows: SM utilization 72%, HBM bandwidth 68% of peak, AllReduce is 22% of step time (appears sequential with backward in nsys trace). Using the 5-step decision tree, diagnose the primary bottleneck and prescribe the exact configuration changes.

Expected answer: Step 1: MFU 38% < 40% → investigate. Step 2: SM utilization 72% → GPU is busy (not overhead-bound). Step 3: HBM bandwidth 68% → not memory-starved, not at ceiling (normal range). Step 4: HBM 68% — not in >80% memory-bound territory. Step 5: AllReduce 22% of step time (> 15% threshold) → COMMUNICATION BOTTLENECK. Furthermore, nsys shows AllReduce appearing AFTER backward (sequential) → gradient overlap is NOT working. Diagnosis: FSDP/DDP gradient AllReduce is not overlapping with backward computation. The AllReduce is sequential, adding 22% overhead to step time. Prescriptions in order: (1) Verify DDP configuration: `DDP(model, bucket_cap_mb=25)` and confirm `backward_prefetch=BackwardPrefetch.BACKWARD_PRE` in FSDP. (2) Check for Python hooks in backward that block NCCL launch — any `loss.backward()` hook that runs synchronous Python code will delay AllReduce. Remove or make async. (3) Verify TP stays within NVLink domain (TP=8 per node) — if TP is crossing IB, those AllReduces will be 10× slower. (4) Run `nccl-tests allreduce_perf -b 128M -e 4G` to verify IB bandwidth. If < 40 GB/s busbw: IB fabric issue. With perfect overlap: step time drops from 22% extra to ~5% → MFU improvement from 38% to ~48%.

Difficulty: Principal Level

Question 3 (Benchmark Validity + Capacity Planning): A vendor presents results showing their new GPU achieves 1.4× the throughput of H100 on Llama-2 70B Offline MLPerf. You must

decide whether to purchase 2,000 of these GPUs. What five validation steps would you take before committing?

Expected answer: (1) Request the MLPerf result artifacts: verify it is the Closed division (Open allows model distillation that inflates numbers), verify the power submission is included (a 1.4× faster GPU drawing 1.6× the power may be worse TCO), verify reproducibility with their published software stack on a loaner unit. (2) Apply the MLPerf blind spots analysis: Offline scenario assumes infinite batching and no latency constraint. Run in Server scenario (Poisson arrivals, P99 latency constraint) — typically 60-70% lower throughput than Offline. The Server scenario is more representative of production. (3) Run your production serving stack (not MLPerf's reference implementation): deploy vLLM or TRT-LLM with your actual production configuration (chunked prefill, continuous batching, INT8 KV cache, prefix caching). Measure throughput and P99 TTFT/TPOT under your traffic distribution. (4) Measure sustained performance: run the benchmark for 8+ hours continuously (thermal steady state, long-session KV degradation). A 1.4× gain at 60 seconds may become 1.1× at 8 hours if there's a thermal or memory management issue. (5) Calculate cost-per-token: GPU purchase price / lifetime useful compute + power cost / throughput. If the new GPU costs 1.7× the H100 and uses 1.3× the power, a 1.4× throughput gain still results in higher cost-per-token. The procurement decision is TCO, not peak throughput.

Difficulty: Principal Level

Question 4 (CI Pipeline + Statistical Significance): Your nightly benchmark CI pipeline shows: baseline mean decode throughput = 11,200 tok/s, std = 180 tok/s (over 20 nights). Tonight's result: 10,700 tok/s. (a) Should you trigger a P1 alert? Show the statistical calculation. (b) What additional checks would you run before concluding it is a real regression? (c) Design the automated profiling response that would run on P1 trigger.

Expected answer: (a) z-score = (10,700 - 11,200) / 180 = -2.78 standard deviations from baseline. Threshold: -2.0σ for P1. -2.78 < -2.0 → EXCEEDS threshold → trigger P1 alert. Absolute change: (11,200 - 10,700) / 11,200 = 4.5% drop. Threshold: > 3% for P1 → also exceeds. Both statistical test and absolute threshold exceeded → P1 alert is justified. (b) Before concluding: (1) Re-run the benchmark 3 times immediately on the same node to distinguish one-time event (thermal spike, background process) from systematic regression. If ≥ 2/3 runs show similar drop → systematic regression. (2) Verify CI environment isolation: check nvidia-smi for other running processes on the benchmark GPU (no co-location), verify GPU temperature at start of benchmark (thermal carryover from previous test?), verify driver version is unchanged. (3) Check if the regression correlates with the PR being tested (revert the PR change on a separate branch and re-run benchmark). If regression disappears → PR introduced the regression. (c) Auto-profiling response on P1: Step 1: nsys profile of 50 training steps (--trace=cuda,nvtx,nccl) → save timeline as artifact. Step 2: Extract top-5 kernels by total execution time from nsys. Step 3: For each of the top-5 kernels, run `ncu --set full --kernel-name X --launch-count 5` → save .ncu-rep as

artifact. Step 4: Diff the ncu output against last-known-good ncu profiles (stored in artifact archive for last 30 days). Step 5: Generate a text summary: "Kernel X changed from Y TFLOPS to Z TFLOPS (diff); HBM bytes changed from A to B (diff); stall type changed from C to D." Post artifact links and summary as a comment on the PR that triggered the CI run. The on-call engineer gets a pre-diagnosed report, not a raw number.

Further Reading

1. *MLPerf Inference Benchmark* — MLCommons (2024). `mlcommons.org/benchmarks/inference` Read the accompanying white paper for each major release, not just the results table. Understanding the measurement protocol is as important as the numbers.
2. *Roofline: An Insightful Visual Performance Model for Multicore Architectures* — Williams et al. (2009). `dl.acm.org/doi/10.1145/1498765.1498785` The foundational paper. The model has not changed; the hardware has.
3. *NVIDIA Nsight Systems User Guide* — NVIDIA (2024). `docs.nvidia.com/nsight-systems/` Focus: CUDA trace, NCCL trace, and the GPU row in the timeline.
4. *NVIDIA Nsight Compute Documentation* — NVIDIA (2024). `docs.nvidia.com/nsight-compute/` Learn to read the roofline chart and the warp stall analysis sections.
5. *Making Deep Learning Go Brrr From First Principles* — Horace He (2022). `horace.io/brrr_intro.html` The best single reference for understanding compute vs memory vs overhead regimes.
6. *Triton Puzzles* — Sasha Rush. `github.com/srush/GPU-Puzzles` Build benchmarking intuition by writing and profiling custom GPU kernels.

— End of Chapter 12 —

AI/ML Infrastructure from Silicon to Scale Venkat Vinjam · Principal Performance Architect © 2026 — All rights reserved